

Exercise 6: Spec-Driven Development with Spec Kit

Session: 6 — Real Development Workflows **Duration:** 30 minutes **Project:** bookstore-web — a new frontend for the BookStore API you already know.

Goal: Run a real spec-driven flow end to end and produce every artifact up to a task breakdown. You will **not** write any application code. Exercise 7 implements what you specify here, with a team of agents working in parallel.

Why a frontend

The BookStore API has no UI. That makes it genuinely unspecified: the HTTP contract is discoverable, but nothing tells you what happens on an empty list, what a failed request looks like, or which fields a book page shows. Those are decisions, and decisions are what a specification is for.

It also breaks into pieces that different agents can build at the same time. That is the property Exercise 7 depends on.

Before you start

Setup is in `preparation.pdf`. Verify it in one command:

```
cd ../bookstore-web && specify check
```

You need three terminals. **Not** the frontend dev server. There is no frontend yet, and that is the point: this session you specify it. Exercise 7 builds it.

Terminal 1 — your backend, from the `bookstore-py` project directory (any of the four backends works; the spec you write is agnostic):

```
python3 main.py
```

Confirm it answers. This must return JSON, not a connection error:

```
curl http://localhost:8080/api/books
```

Terminal 2 — Claude Code, started from `bookstore-web` with read access to your backend:

```
cd ../bookstore-web && claude --add-dir ../bookstore-py
```

Terminal 3 — a plain shell, anywhere in the repo. You need one for `curl` and `git` while Claude Code is busy in terminal 2.

You will call the API on `http://localhost:8080` directly. The finished frontend will reach it at the same-origin path `/api`. `bookstore-web/vite.config.js` proxies that to port 8080, so the paths in your spec are the ones you see here.

Tasks

1. Read the constitution (2 min)

Spec Kit separates two kinds of rules. A **spec** describes one feature and is done when that feature ships. A **constitution** holds the rules that outlive every feature: stack, architecture, what counts as finished. Spec Kit re-reads it at each later step, so a plan or a task that breaks one of these is a defect to fix, not a trade-off to weigh.

Open [`.specify/memory/constitution.md`](#) in `bookstore-web`. Yours is pre-written. You are not authoring one this session.

It has six principles and fits on one page. Read them, then answer this: **What does the constitution say about the API contract, and where does it say the contract comes from?**

Tip: Use Claude Code to find out!

2. `/speckit-specify` — capture intent (3 min)

Use this prompt **exactly as written**:

A web UI for the BookStore API. Users can browse books and open a book to see its details and its author. The API is already running behind `/api`.

It writes `specs/001-*/spec.md`. Open it and scroll to **Assumptions** at the bottom. That is where the spec parks what it invented. Nothing has called the API yet, so every one of them is a guess, and they are the field names and numbers your UI gets built from.

Now read the **Requirements** section in the `spec.md` with one question in mind: **what would someone building this still have to make up?** Three worth checking:

- How does a visitor reach page 2 of a list longer than one screen?
- What does the list show when a page has no books on it?
- What does the page show for a book id that does not exist?

Which of those three does the spec actually answer?

3. `/speckit-clarify` — interrogate the spec (6 min)

This is the core of the exercise. Spec Kit asks up to five targeted questions about what you left underspecified, and writes your answers back into the spec.

Unlike the other commands, this one creates **no new file**. Everything lands in the spec you already have:

```
specs/001-*/spec.md          modified in place, re-saved after
every answer
specs/001-*/checklists/requirements.md  re-validated – only if it exists;
ours will not
```

Two new headings appear, and by design only these two. They are placed near the **top** of the spec, just after its overview section, not appended at the end:

```
## Clarifications

### Session YYYY-MM-DD

- Q: <the question it asked> → A: <the answer you gave>
```

That log is the cheap part. Each answer is *also* applied wherever it belongs: Functional Requirements, User Stories, Data Model, Success Criteria, or Edge Cases. Where an answer contradicts something the spec already said, the old

sentence is **replaced, not added to**. What you get back is not the file you had plus a section at the top.

First, snapshot the spec so you can see all of that later:

```
git add specs/
```

Staging is enough. No commit needed. `git diff` compares your working tree against what you staged, so it will show exactly what `/speckit-clarify` touched. You read that diff in task 4, not now.

Now run `/speckit-clarify`. It asks up to five questions, one at a time, and waits for each answer.

One rule: **if you are guessing, say so in the answer**. Your words go into the spec as written, so the label travels with them. Tasks 4 and 5 come back for these.

Answer fast. Five questions, six minutes. You are making a first draft, not a perfect spec.

Then stop. Do not read the spec yet. That is what you do in task 4, while a slow command runs.

4. `/speckit-plan` — **decide how, and review while it works (6 min)**

This is the slowest command in the flow, and it runs unattended for two to four minutes. That is not dead time. It is when you review what you just wrote.

Start it now, then read on.

It produces **five** files, not one:

<code>specs/001-*/plan.md</code>	the approach
<code>research.md</code>	what it learned about the API
<code>data-model.md</code>	the entities
<code>contracts/</code>	the API contract it extracted
<code>quickstart.md</code>	how to verify

Note the second file. `/speckit-plan` does its own research, unprompted. Task 5 comes back to that.

While it runs: review `specs/001-*/spec.md`

In terminal 3. Do not touch the session running the command:

```
cd ../bookstore-web && git diff specs/001-*/spec.md
```

The `## Clarifications` log at the top tells you what you were *asked*. Only the diff shows **where the answers landed**, and that is the whole point, because `/speckit-clarify` rewrote Functional Requirements, User Stories, Data Model, Success Criteria and Edge Cases in place while you were answering.

Four things to look for:

- **Your marked guesses.** Find each one in the diff. A guess that became a Functional Requirement now looks exactly like an observed fact to everyone downstream, including the agents in Exercise 7.
- **Lines that disappeared.** Look for `-` lines outside the Clarifications block. Where an answer contradicted something the spec already said, the old sentence was *replaced*. Deletions are the edits you never see if you only read the finished file.
- **Answers that travelled further than you expected.** A single reply can rewrite a user story, add an edge case *and* change the data model. Did any of them land somewhere you would not have put them?
- **Anything you now disagree with.** Write it down; do not fix it yet.
`/speckit-plan` is reading this file right now, and editing it mid-run gets you a plan built from two different specs.

Now compare with your neighbour. Put their `spec.md` next to yours and find **one requirement that differs**. You started from the same prompt, against the same API. Would either of you have noticed that difference if neither had written a spec?

5. Check the plan against reality (4 min)

`/speckit-plan` wrote its own account of what this API does, in two places:

```
specs/001-*/research.md  
specs/001-*/contracts/
```

Nobody called the API to write those. They were inferred, and Exercise 7 builds against them.

So call it yourself. In terminal 3, thirty seconds. This answers the first of the three gaps from task 2, plus one thing the spec never thought to ask:

```
# how do you get page 2?
curl -s 'http://localhost:8080/api/books?page=0&size=3'
curl -s 'http://localhost:8080/api/books?page=1&size=3'

# what shape is one book?
curl -s http://localhost:8080/api/books/1
```

Page 0 does what you decided in Session 2: page 1 again, or an error. Either way, pages start at 1. And **a book is not a book:** `GET /api/books/1` returns `{"book": {...}, "author": {...}}`, a wrapper, while the list returns a bare array. Neither is guessable, and neither was in your spec.

Now open both files and check them against what you just saw:

- **Shape** — do they say `GET /api/books/{id}` returns a book? The most likely error of the three, because every REST API the model has ever read returns the resource itself.
- **Paging** — do they say pages start at 1, and what page 0 does? Anything starting at 0 is wrong.
- **Inventions** — endpoints, fields or status codes that appear nowhere in your curl output. A contract is where a plausible invention does the most damage.

Did you find one? Write it down. In a real project it goes back into the plan before anyone writes code.

6. `/speckit-tasks` — break it down (4 min)

Open `specs/001-*/tasks.md`. This is the artifact Exercise 7 consumes, so read it properly.

The task format is `[ID] [P?] [Story]`:

- `[P]` — can run in parallel: different files, no dependencies
- `[US1]`, `[US2]` — which user story the task belongs to

Find the **Parallel Opportunities** and **Parallel Team Strategy** sections at the bottom. That is a staffing plan for agents.

Now check one thing: pick two [P] tasks from **different** user stories. Do they write the same file? Shared things, such as the API client, the stylesheet, or `index.html`, belong to the foundational phase (constitution principle IV). If one lives inside a story, the [P] is a lie and two agents will collide.

7. `/speckit-analyze` — validate (3 min)

A read-only consistency check across `spec.md`, `plan.md` and `tasks.md`. It writes no files.

Read the report and pick **one finding you agree with** and **one you do not**. Be ready to say why.

Two places to look first: the answers you marked as guesses in task 3, and whatever you noted in task 5. If something you *know* to be wrong is absent from this report, that is the lesson. Consistency is not correctness.

8. Commit

```
git add specs/ && git commit -m "spec: bookstore-web frontend"
```

Exercise 7 starts from this. A spec that lives only in a chat window is not a spec.

Pair Discussion (2 min)

- Which `/speckit-clarify` question did you not see coming?
 - Which of your answers travelled further into the spec than you expected?
 - Did `/speckit-plan` get anything wrong about the API? Would `/speckit-analyze` ever have told you?
 - You started from the same prompt against the same API. Where do your two specs differ, and which of those differences would ever have surfaced if neither of you had written one?
 - Could two agents genuinely take your two user stories from `tasks.md` right now? If not, what is in the way?
-

What you should have

```
bookstore-web/specs/001-*/  
  spec.md          intent, clarified  
  plan.md          approach  
  research.md       what the plan learned about the API  
  data-model.md     entities  
  contracts/        API contract  
  quickstart.md     verification  
  tasks.md          the work queue, with [P] markers
```

No application code. That is the point. `/speckit-implement` is Exercise 7.